

## LANGGRAPH MASTERY SERIES



### PHASE 1

# Agentic AI Foundations

Shift your mental model from chains to stateful loops.  
Understand what an agent really is — before you write one.

#### WHAT YOU WILL MASTER

- What an agent is: the LLM + tools + memory loop
- The ReAct pattern: Reason -> Act -> Observe
- Why graphs beat chains for real agents
- The three kinds of agent memory
- How LLMs call functions at runtime
- Agentic thinking vs. chain-based thinking

---

A from-first-principles guide. Hands-on project: build a ReAct agent in pure Python.

Built on the official LangGraph documentation ([docs.langchain.com](https://docs.langchain.com)) · LangGraph v1.1, 2026

# Before the framework: how to think about agents

LangGraph is a tool for building agents. But a tool only helps if you already understand the shape of the thing you are building. This first guide deliberately contains very little LangGraph. Instead it rewires the mental model you bring to the problem, because the single biggest mistake newcomers make is to treat an agent like a longer chain. It is not. An agent is a **loop**, and that one difference changes everything about how you design, debug, and reason about the system.

By the end of this phase you will be able to explain, on a whiteboard and in code, exactly what happens between the moment a user asks a question and the moment an agent answers it — including every decision point, every tool call, and every piece of state that has to survive from one step to the next. The hands-on project builds a working ReAct agent in nothing but Python and a model client, so that when you reach Phase 2 every LangGraph primitive will feel like a name for something you already built by hand.

## How to read this series

There are four phases. Phase 1 (this document) is conceptual. Phase 2 turns these concepts into working LangGraph code. Phase 3 composes agents into larger architectures. Phase 4 ships them to production. Each phase ends with hands-on projects you can run.

## 1. What is an agent?

Strip away the marketing and an agent is a strikingly simple idea. It is a language model that runs in a loop, deciding for itself which actions to take, taking them through tools, observing the results, and repeating until it judges the task complete. The official LangGraph documentation frames it precisely: an agent is *an LLM performing actions using tools, operating in continuous feedback loops, used in situations where problems and solutions are unpredictable*.

Three ingredients combine to make this work. Remove any one and you no longer have an agent:

- **A model (the reasoning engine).** The LLM decides what to do next. It is not just producing prose; it is choosing between actions, given everything it has seen so far.
- **Tools (the hands).** Functions the model can invoke to affect or query the outside world — search the web, run a query, send an email, call an API, do arithmetic reliably.
- **A loop with memory (the working surface).** The results of each action are fed back in, so the next decision is informed by the last observation. Without the loop, the model acts once and stops; with it, the model can course-correct.

## The one-sentence definition

An agent = an LLM + tools + a feedback loop, where the model itself decides when to act, what to act with, and when it is finished. The autonomy lives in that loop.

Contrast this with a plain LLM call. A plain call is a function: text in, text out, once. It cannot look something up, cannot retry, cannot break a hard task into smaller pieces it tackles in sequence. An agent can do all of these because it is allowed to keep going and to use tools between thoughts. That capacity to **keep going** is the whole game.

### A worked example of the loop

Suppose a user asks: *“Find the most popular wireless headphones right now and check if they’re in stock.”* A capable agent reasons and acts in turns, exactly as the LangGraph docs illustrate the ReAct loop:

- 1 Reason:** “Popularity is time-sensitive — I should use the search tool rather than guess.”
- 2 Act:** call `search_products("wireless headphones")`.
- 3 Observe:** the tool returns a ranked list; the top item is model WH-1000XM5.
- 4 Reason:** “I have the most popular item, but I still need to confirm stock before answering.”
- 5 Act:** call `check_inventory("WH-1000XM5")`.
- 6 Observe:** the tool returns “10 units in stock.”
- 7 Reason:** “I now have both facts. I can answer.” The loop ends and the agent replies.

Notice that the number of steps was not fixed in advance. The agent decided it needed two tool calls. A different question might need none, or five. That variability is the signature of an agent and the reason a fixed pipeline cannot express it.

## 2. The ReAct pattern: Reason -> Act -> Observe

ReAct is shorthand for **Reasoning + Acting**. It is the canonical control loop for agents and the pattern LangGraph’s prebuilt agent implements. The idea is to interleave short bursts of natural-language reasoning with concrete tool actions, then feed each observation back into the next round of reasoning. The documentation describes it as *alternating between brief reasoning steps and targeted tool calls, feeding the resulting observations into subsequent decisions until the agent can deliver a final answer*.

### The three phases, in detail

- **Reason.** Given the conversation so far and the latest observation, the model thinks about what is still missing and what the best next action is. This is where the model can catch its own mistakes — if the last tool returned an error, it reasons about a fallback.
- **Act.** The model emits a structured request to call a specific tool with specific arguments. This is not free-text; it is a machine-readable tool call (more on the mechanics in section 5).
- **Observe.** The runtime executes the tool and appends the result to the conversation as a tool message. The model now has a new fact to reason about, and the loop repeats.

The loop terminates when the model produces a normal answer with no tool call — its way of signalling “I’m done” — or when a guardrail (an iteration cap) forces a stop. Designing good termination conditions is a recurring theme across all four phases.

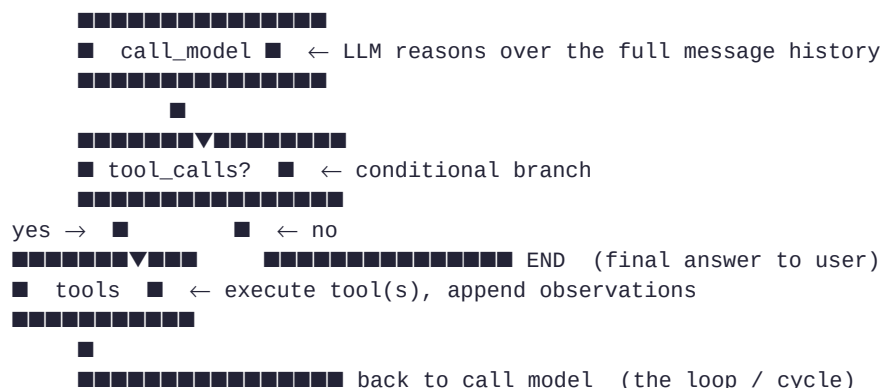
### Why interleaving matters

Forcing the model to reason in the open before each action makes its behaviour far easier to inspect and debug. You can read the chain of thoughts and tool calls and see exactly where a wrong turn was taken. A model that acts without visible reasoning is a black box; ReAct cracks it open.

## ReAct as a state machine

It helps to picture ReAct as a tiny state machine with two real states and a decision in between:

*The ReAct loop drawn as nodes and a cycle. Phase 2 builds this exact shape in LangGraph.*



Hold this diagram in your mind. In Phase 2 you will see that LangGraph's prebuilt agent is literally this graph: a model node, a tools node, and a conditional edge that loops back or finishes. Everything else is convenience.

## 3. Why graphs over chains?

Early LLM applications were built as **chains**: linear pipelines where the output of one step feeds the input of the next, the flow always moves forward, and the program terminates at the end. In graph terms a chain is a **Directed Acyclic Graph (DAG)** — directed because data flows one way, acyclic because there are no loops back. Chains are perfect for well-defined, fixed sequences: summarise, then translate, then format.

But real agents need to do things a DAG cannot express. They need to **loop** (try a tool, look at the result, try again), to **branch** on what they discover at runtime, and to carry **persistent state** across many steps. The moment you want a system that iterates, re-evaluates intermediate results, or decides dynamically what to do next, the chain abstraction breaks down.

### The three capabilities a graph unlocks

| Capability | What it means                                                                                                            | Why a chain can't do it                                                            |
|------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Cycles     | A node can route back to an earlier node, so the agent can repeat reason→act→observe as many times as needed.            | A DAG forbids edges that return to earlier nodes — by definition it has no cycles. |
| Branches   | At runtime the agent picks which node runs next based on the current state (e.g. route a refund vs. a returns question). | A fixed pipeline runs the same steps in the same order every time.                 |

| Capability       | What it means                                                                                     | Why a chain can't do it                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Persistent state | A shared, typed state object is threaded through every node and can be checkpointed at each step. | Chains pass data step-to-step but have no first-class, inspectable, resumable state across the whole run. |

LangGraph models your application as a graph for exactly these reasons. The official overview is blunt about its purpose: it is a *low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents*, focused on durable execution, streaming, and human-in-the-loop — the capabilities a DAG cannot provide.

### The mental shift to internalise

Chains answer “what is the next step?” Graphs answer “given the current state, where should we go next, and have we finished?” Agents live in the second question. Stop thinking in pipelines; start thinking in states and transitions.

## State is the backbone

Because the flow can loop and branch, there must be a single source of truth that survives between steps: the **state**. Every node reads the current state and returns an update to it. For a conversational agent the most important piece of state is the list of messages — the running transcript of human turns, AI turns, tool calls, and tool results. Keeping a faithful, ordered message history is what lets the model reason coherently across many loop iterations. Phase 2 makes this concrete with typed state and reducers; for now, just hold the principle: **the graph computes, the state remembers**.

## 4. Types of agent memory

“Memory” is an overloaded word. For agents it helps to separate it into three distinct layers, each solving a different problem and living in a different place.

### 4.1 In-context (working) memory

This is the conversation that lives inside the model's context window right now — the messages the model can actually see when it reasons. It is fast, immediate, and the model uses it without any retrieval step. But it is bounded: context windows are finite, so a very long conversation eventually must be trimmed or summarised. In-context memory is also **short-term** by nature — it disappears when the session ends unless you persist it.

- **Holds:** the active message history and any scratch fields the agent is tracking this turn.
- **Lifetime:** the current run / thread, while it fits in the window.
- **Risk:** overflow. Manage it with trimming, summarisation, or selective context engineering.

### 4.2 External (long-term) memory

To remember things **across sessions** — a user's name, their stated preferences, facts learned in a previous conversation — you need storage that lives outside any single context window. In LangGraph this is the **store**: a key-value memory, namespaced (typically per user), that any node can write to and search. Crucially, it is shared across threads, so a fact saved in one conversation can be recalled in a completely different one.

*The shape of external memory (LangGraph Store). Phase 2 wires this into a real agent.*

```
# Long-term memory is namespaced – usually per user – and persists across threads
namespace = (user_id, "memories")
store.put(namespace, memory_id, {"food_preference": "I like pizza"})

# Later, in a different conversation (different thread, same user):
memories = store.search(namespace, query="What does the user like to eat?")
```

- **Holds:** durable facts, preferences, summaries, embeddings for semantic recall.
- **Lifetime:** indefinite — survives restarts and spans every conversation.
- **Backed by:** in-memory for dev; Postgres / Redis for production.

### 4.3 Episodic memory

Episodic memory is the record of **what happened** — the sequence of past steps, decisions, and outcomes. In LangGraph this emerges naturally from **checkpoints**: a snapshot of the graph state saved at every step, organised into threads. Because each step is checkpointed, the full history of a run is available — you can inspect it, resume from it, or even rewind to an earlier checkpoint and explore a different path (time travel). This is the substrate that makes human-in-the-loop, fault tolerance, and debugging possible.

| Memory type          | Scope                | Backed by (LangGraph)                   | Primary use                      |
|----------------------|----------------------|-----------------------------------------|----------------------------------|
| In-context / working | Current turn         | Message state in the context window     | Coherent reasoning now           |
| External / long-term | Across all sessions  | Store (namespaced KV, optional vectors) | Recall facts & preferences       |
| Episodic             | A run's full history | Checkpoints / threads                   | Resume, debug, time-travel, HITL |

#### Why this taxonomy matters

When something goes wrong, the first useful question is “which memory failed?” Did the model lose the thread because context overflowed (working memory)? Did it forget a known preference (external memory)? Could you not reproduce the bug because the run wasn't checkpointed (episodic memory)? Naming the layers tells you where to look.

## 5. Tool use: how LLMs call functions at runtime

Tools are how an agent reaches beyond text into the world. A tool is just a function with a name, a description, and a typed signature. The mechanism that lets a model use one is **tool calling** (sometimes “function calling”), and understanding it removes most of the mystery from agents.

### The mechanics, step by step

- 1 **You bind tools to the model.** You hand the model a list of tools. Behind the scenes their names, descriptions, and argument schemas are passed to the model so it knows what is available and how to call each one.

- 2 The model emits a tool call, not prose.** When the model decides a tool is needed, it does not write “please run search”. It returns a structured **tool call**: the tool's name plus a JSON object of arguments matching the schema. The model never runs the tool itself.
- 3 Your runtime executes the tool.** Your code (or LangGraph's tool node) reads the tool call, invokes the real Python function with those arguments, and captures the return value.
- 4 The result is fed back as a tool message.** The observation is appended to the message history as a tool result, tagged with the call's id. Now the model can reason about it.
- 5 Repeat.** The model may call more tools or produce a final answer. This is the ReAct loop viewed from the tool's perspective.

*A tool, bound to a model. The model proposes the call; your runtime performs it.*

```
from langchain.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply `a` and `b`.""" # the docstring becomes the tool description
    return a * b

# Bind the tool so the model knows it exists and how to call it
llm_with_tools = llm.bind_tools([multiply])

msg = llm_with_tools.invoke("What is 2 times 3?")
print(msg.tool_calls)
# [{'name': 'multiply', 'args': {'a': 2, 'b': 3}, 'id': 'call_...'}]
# Note: the model REQUESTED the call. Your code still has to run multiply() and
# return 6 back to the model before it can answer.
```

### The crucial separation

The model decides WHICH tool to call and WITH WHAT arguments. Your code decides HOW the tool runs and feeds the result back. The model proposes; the runtime disposes. Keeping these responsibilities separate is what makes agents controllable and safe.

Good tool design is therefore mostly good **description** design. The model chooses tools based on their names and docstrings, so a clear, specific description (“Search the product catalog by keyword; returns up to five ranked matches”) leads to better decisions than a vague one (“search stuff”). Tool descriptions are prompts in disguise.

### Too many tools hurt

A model handed twenty overlapping tools makes worse choices than one given three well-scoped ones. Overloading the toolset bloats context and increases errors. When you find yourself with too many tools, that is a signal you may need the multi-agent patterns of Phase 3 — or simply fewer, sharper tools.

## 6. Agentic vs. chain-based thinking

Everything above converges on a single shift in how you design systems. The table below contrasts the two mindsets so you can feel the difference in your bones.

| Dimension    | Chain-based thinking           | Agentic thinking                |
|--------------|--------------------------------|---------------------------------|
| Control flow | Fixed and predetermined by you | Decided at runtime by the model |

| Dimension       | Chain-based thinking           | Agentic thinking                            |
|-----------------|--------------------------------|---------------------------------------------|
| Shape           | Linear DAG (forward only)      | Graph with cycles and branches              |
| Number of steps | Known in advance               | Varies per input; the agent decides         |
| Tool use        | Optional, scripted             | Core mechanism, chosen dynamically          |
| State           | Passed step to step            | Shared, typed, checkpointed across the run  |
| Error recovery  | Usually fails the pipeline     | Can observe an error and retry or fall back |
| Best for        | Well-defined, repeatable tasks | Open-ended, unpredictable tasks             |

Neither is universally better. The discipline is choosing correctly. The LangGraph documentation draws the line cleanly: **workflows** have predetermined code paths and operate in a fixed order, while **agents** are dynamic and define their own processes and tool usage. A great deal of production engineering is deciding, for each sub-problem, whether you want the predictability of a workflow or the adaptability of an agent — and LangGraph lets you mix both in one graph.

#### A practical heuristic

If you can write down the exact steps ahead of time and they never change, build a workflow — it is cheaper, faster, and easier to test. If the right steps depend on what you discover along the way, build an agent. Most real systems are a workflow skeleton with agentic nodes inside it.



# Hands-on: build a ReAct agent in pure Python

## PROJECT 1

### A from-scratch ReAct calculator agent — no LangGraph, no helpers

**Goal:** Implement the Reason -> Act -> Observe loop by hand so every LangGraph primitive in Phase 2 feels familiar. You will write the loop, the tool dispatch, and the termination logic yourself.

This project deliberately uses no framework. The point is to feel the loop in your own code so that when LangGraph gives you a `ToolNode` and a conditional edge, you recognise them as conveniences for things you already built. We build a small agent that can do exact arithmetic by calling Python tools rather than trusting the model's mental math.

### Step 1 — Define tools and bind them

```
import os
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage

os.environ["ANTHROPIC_API_KEY"] = "sk-..." # your key
llm = init_chat_model("claude-sonnet-4-6")

@tool
def add(a: float, b: float) -> float:
    """Add two numbers and return the exact sum."""
    return a + b

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers and return the exact product."""
    return a * b

@tool
def divide(a: float, b: float) -> float:
    """Divide a by b and return the exact quotient."""
    return a / b

tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}
llm_with_tools = llm.bind_tools(tools)
```

### Step 2 — Write the loop yourself

This is the heart of the project. Read it slowly: it is the ReAct state machine from section 2, expressed as a while loop. The message list is your working memory; the loop is your graph; the `if last.tool_calls` check is your conditional edge.

```
def run_agent(question: str, max_steps: int = 6) -> str:
    # Working memory: the running transcript the model reasons over.
    messages = [
        SystemMessage("You are a careful calculator. Use the tools for ALL arithmetic; "
                       "never compute in your head. When you have the final number, state it plainly."),
        HumanMessage(question),
    ]
```

```

for step in range(max_steps):
    ai = llm_with_tools.invoke(messages)
    messages.append(ai)

    if not ai.tool_calls:
        return ai.content

    for call in ai.tool_calls:
        fn = tools_by_name[call["name"]]
        result = fn.invoke(call["args"])
        # OBSERVE: feed the result back, tagged with the call id
        messages.append(ToolMessage(content=str(result), tool_call_id=call["id"]))

    return "Stopped: hit the step limit before finishing."

print(run_agent("What is (12 + 8) multiplied by 3, then divided by 4?"))

```

### Step 3 — Trace what happens

Run it and watch the loop turn. For the example question the agent typically: (1) reasons it should add first, calls `add(12, 8)` -> observes 20; (2) reasons it should multiply, calls `multiply(20, 3)` -> observes 60; (3) calls `divide(60, 4)` -> observes 15; (4) produces no tool call and answers “15”. The number of iterations was chosen by the model, not by you — that is the agent's autonomy at work.

- Add a `print(step, ai.tool_calls)` inside the loop to see every decision. This is your first taste of **tracing**, formalised in Phase 4 with LangSmith.
- Try a question needing no tools (“Say hello”). The loop returns on the first pass — the termination logic works in both directions.
- Ask something the tools cannot do (“What’s the weather?”). Watch how the model behaves without a matching tool — a useful lesson in tool coverage.

### What you just built (and what comes next)

You implemented, by hand, every concept in this guide: a tool-calling model, a reason/act/observe loop, working memory as a message list, a conditional “am I done?” edge, and a termination guardrail. In Phase 2, LangGraph will give you named, durable, streamable versions of each of these — `StateGraph`, `ToolNode`, conditional edges, checkpoints — but the skeleton will be identical to the loop you just wrote.

#### Phase 1 checkpoint

You can now (1) define an agent as LLM + tools + loop, (2) explain ReAct as Reason->Act->Observe, (3) justify graphs over chains via cycles, branches, and state, (4) distinguish in-context, external, and episodic memory, and (5) describe tool calling end to end. You are ready for Core LangGraph.

**Coming in Phase 2 — Core LangGraph:** `StateGraph`, typed state with `TypedDict` and `Pydantic`, node functions, conditional edges, `ToolNode` and `bind_tools`, checkpoints (`MemorySaver`, `SQLite`, `Postgres`), human-in-the-loop with interrupts, and streaming. You will rebuild this ReAct agent the LangGraph way, then add persistent memory and a human-approval step.